

Algorithms, AI & Data Science II

Prof. Dr. Ingo Scholtes

Chair of Machine Learning for Complex Networks
Center for Artificial Intelligence and Data Science (CAIDAS)
Julius-Maximilians-Universität Würzburg

ingo.scholtes@uni-wuerzburg.de

Notes:

- **Lecture L20: Flow Maximization in Graphs** 16.07.2025
- **Educational objective:** We motivate the study of flows in graphs and show how we can compute maximum flows based on the Ford-Fulkerson algorithm. We further explore the relationship between flow maximization and cut minimization.
 - Flows in graphs
 - Flow Maximization with the Ford-Fulkerson Method
 - Max-Flow Min-Cut Theorem
- **Handout version:** 2025/07/09 16:34:22

Motivation

- ▶ we explored **cluster detection** algorithms and **minimal edge cuts** in graphs
- ▶ general problem: find “bottlenecks” in the topology
- ▶ we often use graphs as models for **distribution networks**
 - ▶ transportation and logistic networks
 - ▶ networks of water/gas pipes
 - ▶ electrical distribution network
- ▶ consider **flow of quantities/commodities** along edges from a source node to a sink node

questions

- ▶ how can we **maximize flow** from source to sink?
- ▶ how can we **optimally disrupt flows** from source to sink?

→ TE Harris, FSRoss, 1955

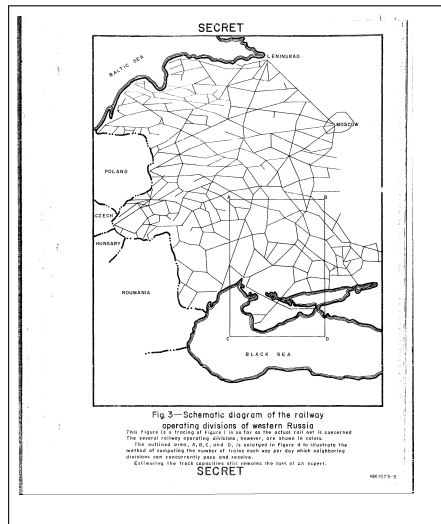


image credit: adapted from → TE Harris, FSRoss, 1955

Notes:

Applications

- ▶ problem first proposed in the context of war logistics, i.e. how to **disrupt flow of war material from factories to the front** along railway networks → TE Harris, FSRoss, 1955
- ▶ various practical problems can be addressed based on **maximization of flows in graphs**

examples

- ▶ image segmentation
- ▶ planning of communication networks
- ▶ design of electronic circuits
- ▶ maximum matchings in bipartite networks
- ▶ baseball league elimination

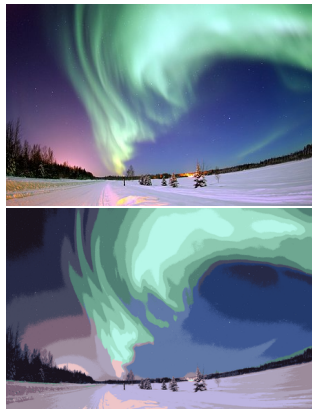


image segmentation

image credit: public domain

Notes:

Flow Networks

- ▶ let $G = (V, E)$ be directed graph where edges $(v, w) \in E$ have **non-negative capacities** $c(v, w) \geq 0$
- ▶ for $v, w \in V$ we define $c(v, w) := 0$ iff $(v, w) \notin E$
- ▶ for **source node** $s \in V$ and **sink node** $t \in V$ we call $f : V \times V \rightarrow \mathbb{R}$ **flow on G** iff

1. f respects **edge capacity constraints**, i.e.

$$\forall u, v \in V : f(u, v) \leq c(u, v)$$

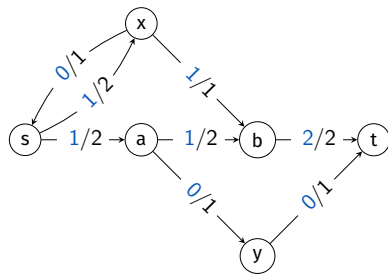
2. f fulfils **skew symmetry**, i.e.

$$\forall u, v \in V : f(u, v) = -f(v, u)$$

3. f fulfils **flow conservation**, i.e.

$$\forall u \in V - \{s, t\} : \sum_{v \in V} f(u, v) = 0$$

- ▶ we call graph with edge capacities c and flow f a **flow network**



flow network where label $1/2$ indicates flow $f(u, v) = 1$ on edge (u, v) with capacity 2

negative flows due to skew symmetry are omitted

Notes:

Net flows

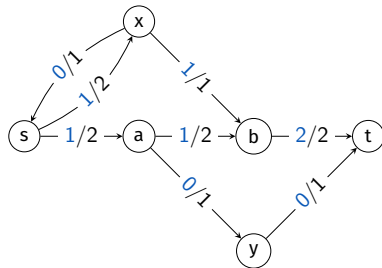
- ▶ $f(u, v)$ = **positive or negative net flow** from u to v
 - ▶ $f(u, v) > 0$ indicates **flow from u to v**
 - ▶ $f(u, v) < 0$ indicates **flow from v to u**
- ▶ $(u, v) \notin E \wedge (v, u) \notin E \implies f(u, v) = f(v, u) = 0$
- ▶ **total net flow into vertex v** can be calculated as

$$\sum_{u \in V} f(u, v)$$

- ▶ **total net flow out of vertex v** can be calculated as

$$\sum_{w \in V} f(v, w)$$

- ▶ due to flow conservation/skew symmetry, we have **zero total net flow into and out of $v \in V - \{s, t\}$**



example

- ▶ total net flow **into t**
 $\sum_{u \in V} f(u, t) = 2 + 0 = 2$
- ▶ total net flow **out of s**
 $\sum_{w \in V} f(s, w) = 1 + 1 = 2$
- ▶ total net flow **into a**
 $\sum_{u \in V} f(u, a) = 1 - 1 - 0 = 0$
- ▶ total net flow **out of b**
 $\sum_{w \in V} f(b, w) = 2 - 1 - 1 = 0$

Notes:

Net flow summation and flow value

- ▶ for flow $f : V \times V \rightarrow \mathbb{R}$ and $X, Y \subseteq V$ we define

$$f(X, Y) := \sum_{x \in X} \sum_{y \in Y} f(x, y)$$

i.e. **sum net flows** from all nodes in X to all nodes in Y

- ▶ for $u \in V$ define $f(u, X) := f(\{u\}, X)$ and $f(X, u) := f(X, \{u\})$

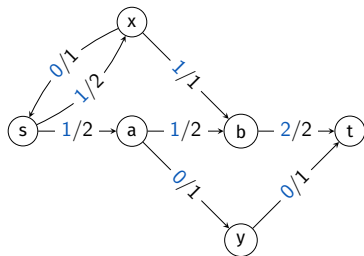
- ▶ **flow conservation** property can be expressed as

$$\forall u \in V - \{s, t\} : f(u, V) = 0$$

- ▶ for flow $f : V \times V \rightarrow \mathbb{R}$ **flow value** $|f|$ is defined as

$$|f| := f(s, V)$$

i.e. **total net flow out of source node s**



properties

→ self-study questions

For flow f and sets $X, Y \subseteq V$ we have:

- ▶ $f(X, X) = 0$
- ▶ $f(X, Y) = -f(Y, X)$

For flow f and sets $X, Y, Z \subseteq V$ with $X \cap Y = \emptyset$ we have

- ▶ $f(X \cup Y, Z) = f(X, Z) + f(Y, Z)$
- ▶ $f(Z, X \cup Y) = f(Z, X) + f(Z, Y)$

Notes:

Group Exercise 20-01

- Let $G = (V, E)$ be a directed graph with flow f . Show that the flow value $|f|$ corresponds to the total net flow into the sink node t , i.e.

$$|f| = f(V, t)$$

We first note that, by definition, the flow value $|f|$ corresponds to the total net flow out of the source node s .

$$\begin{aligned} |f| &:= f(s, V) \\ &= f(V, V) - f(V \setminus \{s\}, V) \\ &= f(V, V \setminus \{s\}) \\ &= f(V, t) + f(V, V - \{s, t\}) \\ &= f(V, t) \quad \square \end{aligned}$$

Notes:

Flow Maximization

- ▶ given graph with edge capacities, source node s and sink node t , how can we find flow f with maximum value $|f|$?
- ▶ **Ford and Fulkerson** proposed greedy maximization that builds on three key concepts → Ford & Fulkerson, 1956
 - ▶ **residual network**, which represents capacities currently not used by flows
 - ▶ **augmenting path** from source to sink along which flow can be increased
 - ▶ **minimal cut**, which allows to prove that flow is maximized when no augmenting path exists
- ▶ basis for **algorithmic implementations** that determine next augmenting path
- ▶ basis for **max-flow min-cut theorem** that relates minimal cuts and maximum flows



Delbert Ray Fulkerson, 1924–1976

image credit: Wikimedia Commons, CC-BY-SA

Ford-Fulkerson approach

```
procedure FORD_FULKERSON( $G, s, t$ )  
   $f \leftarrow \text{zero\_flow}()$   
  while augmenting_path( $G, f$ )  $\neq$  Null do  
     $p \leftarrow \text{augmenting\_path}(G, f)$   
     $f \leftarrow \text{augment\_flow}(G, f, p)$   
  end while  
  return  $f$   
end procedure
```

Notes:

Augmenting paths

- ▶ given graph G and flow f , an **augmenting path** is a path $p = (v_0, v_1, v_2, \dots, v_l)$ from $v_0 = s$ to $v_l = t$ iff
 - ▶ $(v_i, v_{i+1}) \in E$ for $i = 0, \dots, l-1$, i.e. p is path in G
 - ▶ flow along path can be increased, i.e.

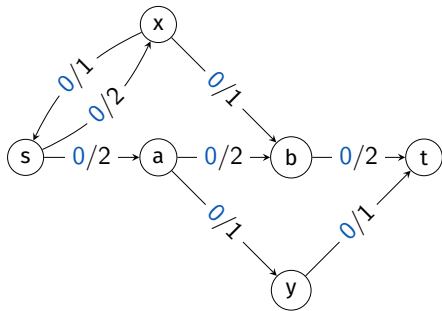
$$\min_{i \in \{0, \dots, l-1\}} c(v_i, v_{i+1}) - f(v_i, v_{i+1}) > 0$$

- ▶ we define **value $|p|$ of augmenting path p** as

$$|p| := \min_{i \in \{0, \dots, l-1\}} c(v_i, v_{i+1}) - f(v_i, v_{i+1})$$

i.e. amount by which flow f can be increased along p

- ▶ augmenting path exists for flow $f \implies |f|$ not maximal
- ▶ but is $|f|$ **maximal if no augmenting path exists?**



example

Augmenting paths:

- ▶ $p_1 = (s, a, b, t)$
- ▶ $p_2 = (s, x, b, t)$
- ▶ $p_3 = (s, a, y, t)$

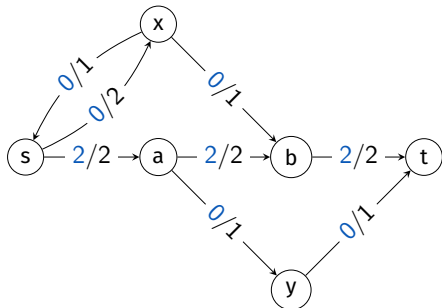
Their values are:

- ▶ $|p_1| = 2$
- ▶ $|p_2| = 1$
- ▶ $|p_3| = 1$

Notes:

Greedy flow maximization?

- ▶ start with zero flow, i.e. $f(u, v) = 0 \quad \forall u, v \in V$
- ▶ we can find augmenting path $p_1 = (s, a, b, t)$ with $|p_1| = 2$
- ▶ we can use this augmenting path to construct new flow f' with $|f'| = 2$
 - ▶ can you find another augmenting path?
 - ▶ does flow f' have maximal value?
- ▶ naive calculation of augmenting path in flow network can lead to **local optimum**
- ▶ how can we avoid such local optima?



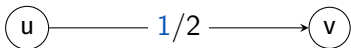
$$|f'| = 2$$

Notes:

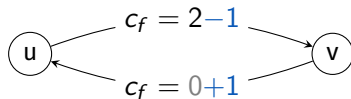
Residual network

- ▶ to avoid local optima, we **change the graph in which we calculate augmenting path**
- ▶ for graph G with capacities c and flow f , we construct **residual network** G_f where for $u, v \in V$ capacities c_f are defined as

$$c_f(u, v) := c(u, v) - f(u, v) \quad \text{and (due to skew symmetry)} \quad c_f(v, u) := c(v, u) + f(u, v)$$



original **flow network**

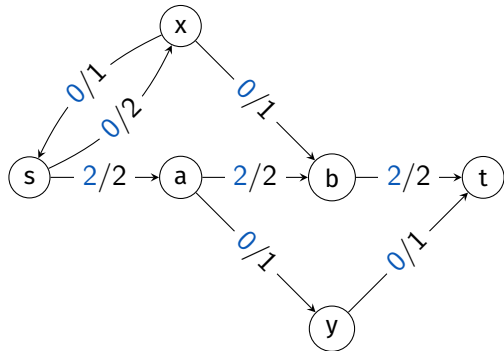


constructed **residual network**

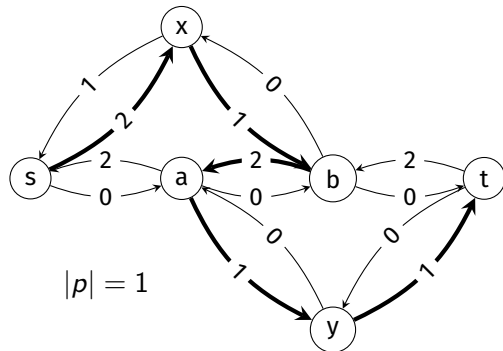
Notes:

Local optimum?

Using the residual network, let us reconsider the problem from before:



original **flow network**



residual network

we can find **augmenting path in residual network** that captures “rerouting” of flows
(allowing to decrease flow from *a* to *b*)

Notes:

Ford-Fulkerson method

- ▶ consider directed graph G with capacities c , source node s and sink node t
- ▶ initially set zero flow f , i.e. $\forall u, v \in V : f(u, v) = 0$
- ▶ construct residual network G_f
- ▶ while **augmenting path** $p = (v_0, \dots, v_l)$ **exists in residual network** G_f , augment flow as follows
 1. if (v_i, v_{i+1}) points **in the direction** of f , **increase** flow of (v_i, v_{i+1}) by $|p|$
 2. if (v_i, v_{i+1}) , points **in opposite direction** of f , **decrease** flow of (v_i, v_{i+1}) by $|p|$
- ▶ algorithm terminates **if no augmenting path exists** in residual network G_f



Delbert Ray Fulkerson, 1924–1976

image credit: Wikimedia Commons, CC-BY-SA

Ford-Fulkerson approach

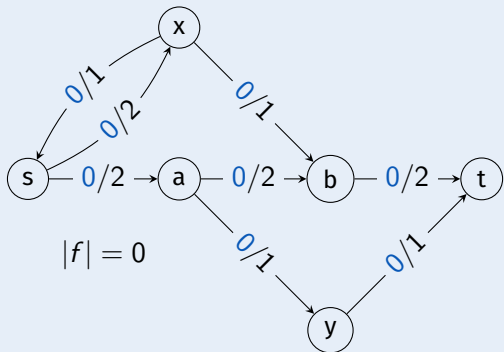
```
procedure FORD_FULKERSON( $G, s, t$ )  
   $f \leftarrow \text{zero\_flow}()$   
   $G_f \leftarrow \text{residual\_network}(G, f)$   
  while augmenting_path( $G_f$ )  $\neq$  Null do  
     $p \leftarrow \text{augmenting\_path}(G_f)$   
     $f \leftarrow \text{augment\_flow}(G, f, p)$   
     $G_f \leftarrow \text{residual\_network}(G, f)$   
  end while  
  return  $f$   
end procedure
```

Notes:

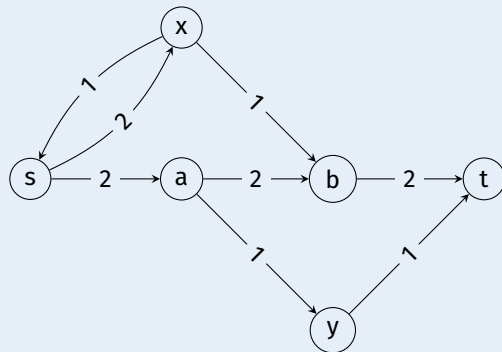
Group Exercise 20-02

1/7

- Run the Ford-Fulkerson method on the example network from the previous slides and output the maximum flow from source s to sink t .



initial **flow network** with zero flow



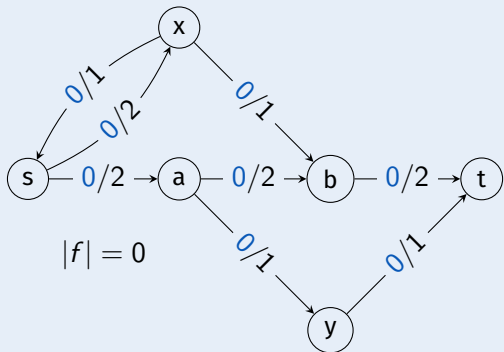
construct **residual network**

Notes:

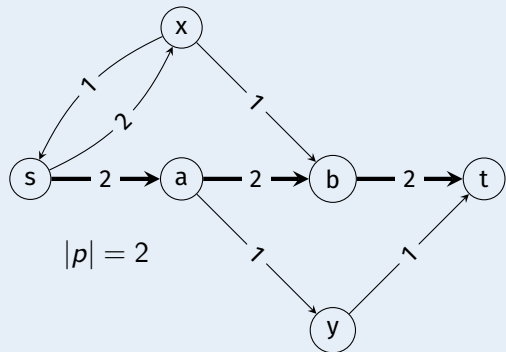
Group Exercise 20-02

2/7

- Run the Ford-Fulkerson method on the example network from the previous slides and output the maximum flow from source s to sink t .



zero **flow network**



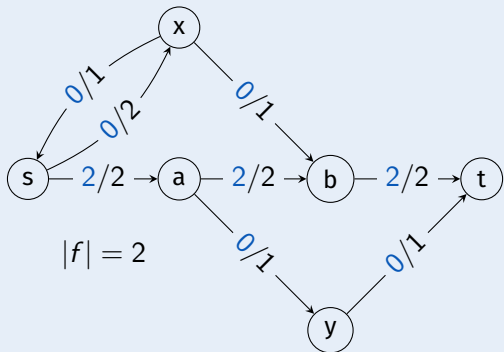
find **augmenting path**

Notes:

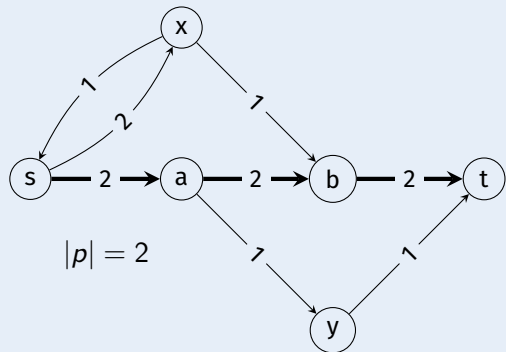
Group Exercise 20-02

3/7

- Run the Ford-Fulkerson method on the example network from the previous slides and output the maximum flow from source s to sink t .



use path to augment **flow**



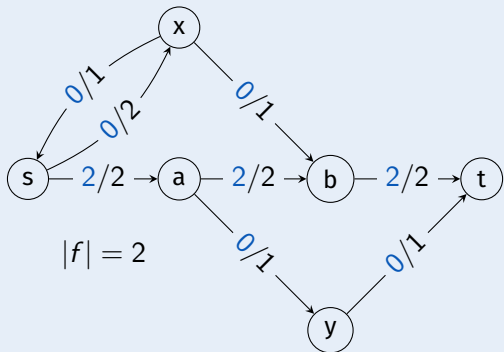
augmenting path

Notes:

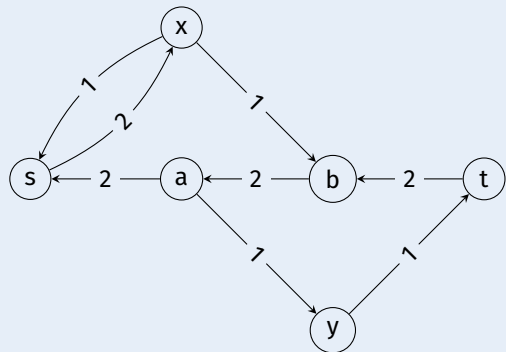
Group Exercise 20-02

4/7

- Run the Ford-Fulkerson method on the example network from the previous slides and output the maximum flow from source s to sink t .



updated **flow network**



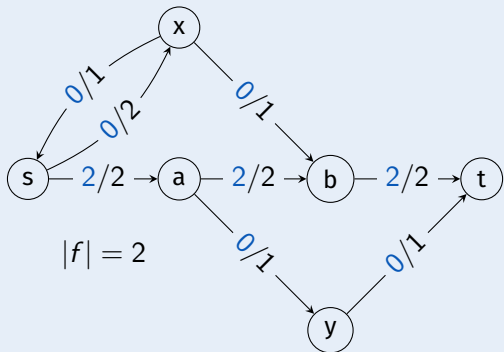
construct **residual network**

Notes:

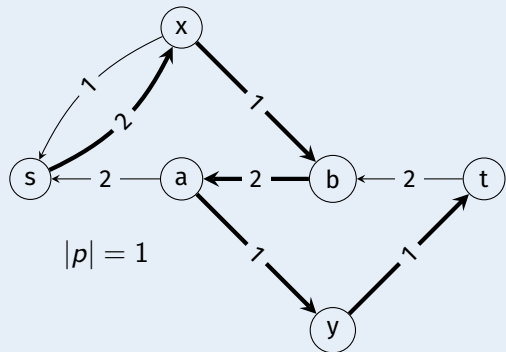
Group Exercise 20-02

5/7

- Run the Ford-Fulkerson method on the example network from the previous slides and output the maximum flow from source s to sink t .



flow network



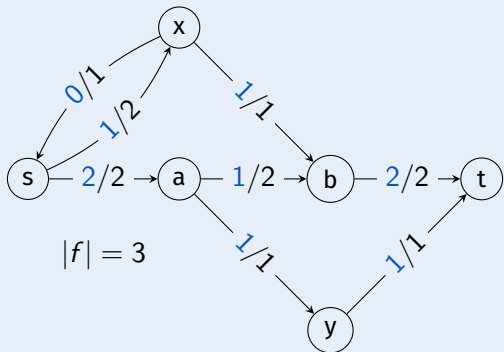
find augmenting path

Notes:

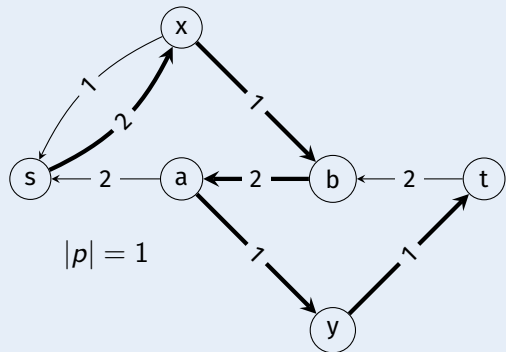
Group Exercise 20-02

6/7

- Run the Ford-Fulkerson method on the example network from the previous slides and output the maximum flow from source s to sink t .



use path to augment **flow**



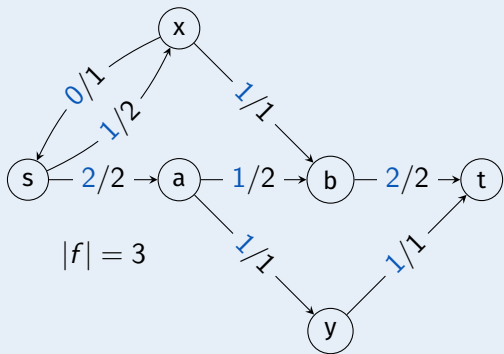
augmenting path

Notes:

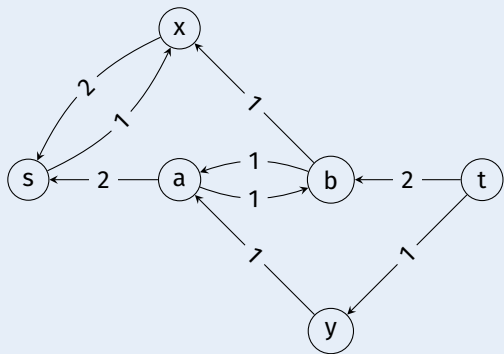
Group Exercise 20-02

7/7

- Run the Ford-Fulkerson method on the example network from the previous slides and output the maximum flow from source s to sink t .



updated **flow network**



construct **residual network**

algorithm terminates since there is **no (augmenting) path from s to t**

Notes:

Ford-Fulkerson algorithm?

- ▶ we applied Ford-Fulkerson method to calculate a maximum flow in our example network
- ▶ we technically do not have an “algorithm” yet that would tell us exactly **how to find the next augmenting path**
- ▶ we further do not know whether the termination of the algorithm yields a flow with **maximum value**
- ▶ to prove correctness of the method we must prove the following theorem

augmenting path theorem

If no augmenting path exists in residual network G_f for flow network G with flow f , then f is maximal flow on G .

- ▶ to prove theorem, we first introduce **minimal cuts**

open questions

- ▶ how can we select the next augmenting path?
- ▶ does termination of algorithm guarantee that found flow is maximal?

Notes:

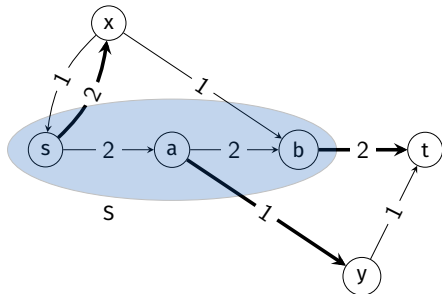
Minimal cuts

- ▶ let $G = (V, E)$ be directed graph with source node s and sink node t
- ▶ $s - t$ **cut** is a partition of nodes in two disjoint sets S and T that contain source and sink, i.e. we have $S, T \subseteq V$
 - ▶ $S \cup T = V$
 - ▶ $S \cap T = \emptyset$
 - ▶ $s \in S, t \in T$

- ▶ for flow network $G = (V, E)$ with capacity c , we define **capacity of $s - t$ cut** as

$$c(S, T) = \sum_{(u,v) \in E, u \in S, v \in T} c(u, v)$$

- ▶ can we find $s - t$ cut with **minimal capacity**?



example

- ▶ $S = \{s, a, b\}$ and $T = \{x, y, t\}$ is an $s - t$ cut of G for source node s and sink node t
- ▶ $c(S, T) = 5$

Notes:

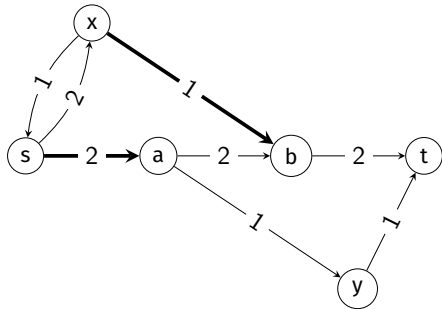
Max-flow min-cut theorem

- ▶ finding maximum flow and minimum cut are **dual problems**
- ▶ this is expressed by following central theorem:

max-flow min-cut theorem → Ford & Fulkerson, 1956

For a flow network G with source s and sink t , the value $|f|$ of the maximum flow f is equal to the capacity $c(S, T)$ of the minimal $s - t$ cut $V = S \cup T$.

- ▶ in the following, we prove the augmenting path theorem and max-flow min-cut theorem simultaneously



example

- ▶ $S = \{s, a, b\}$ and $T = \{x, y, t\}$ is an $s - t$ cut of G for source node s and sink node t
- ▶ $c(S, T) = 5$
- ▶ since f is maximum flow with $|f| = 3$ there exists an $s - t$ cut $V = S \cup T$ with $c(S, T) = 3$
- ▶ $S = \{s, x\}$ and $T = \{a, b, y, t\}$ is an $s - t$ cut of G for source node s and sink node t with $c(S, T) = 3$

Notes:

Outline of proof

- ▶ lemma 1: for any flow network G , any flow f and any $s - t$ cut, the net flow across an $s - t$ cut equals the flow value $|f|$, i.e.

$$|f| = f(S, T)$$

- ▶ lemma 2: for any flow network G , any flow f and any $s - t$ cut $V = S \cup T$, the value $|f|$ of flow f is at most the capacity of the cut, i.e.

$$|f| = f(S, T) \leq c(S, T)$$

- ▶ lemma 3: for any flow network G with source s and sink t , the following statements are equivalent:
 - (1) f is maximum flow in G
 - (2) no augmenting path exists in residual network G_f
 - (3) $|f| = c(S, T)$ for some $s - t$ cut $V = S \cup T$

Relation to theorems

- ▶ If $(2) \Leftrightarrow (1)$ in lemma 3, then $(2) \Rightarrow (1)$, which is the **augmenting path theorem**
- ▶ **max-flow min-cut theorem** follows from
 - ▶ $(1) \Leftrightarrow (3)$ in lemma 3, which implies existence of $s - t$ cut with $c(S, T) = |f|$, and
 - ▶ lemma 2, which implies that there can not be a cut with larger capacity

Notes:

Proof of lemma 1

- lemma 1: for any flow network G , any flow f and any $s - t$ cut, the net flow across an $s - t$ cut equals the flow value $|f|$, i.e.

$$|f| = f(S, T)$$

- proof:

$$\begin{aligned} |f| &= f(s, V) \\ &= f(s, V) + f(S - \{s\}, V) \text{ (flow conservation)} \\ &= f(S, V) \\ &= \underbrace{f(S, S)}_{=0} + f(S, T) \quad (\text{since } V = S \cup T) \\ &= f(S, T) \quad \square \end{aligned}$$

properties

We flow f with source s and sink t we have:

► $|f| = f(s, V)$

► $|f| = f(V, t)$

For flow f and sets $X, Y \subseteq V$ we have:

► $f(X, X) = 0$

► $f(X, Y) = -f(Y, X)$

For flow f and sets $X, Y, Z \subseteq V$ with $X \cap Y = \emptyset$ we have

► $f(X \cup Y, Z) = f(X, Z) + f(Y, Z)$

► $f(Z, X \cup Y) = f(Z, X) + f(Z, Y)$

Notes:

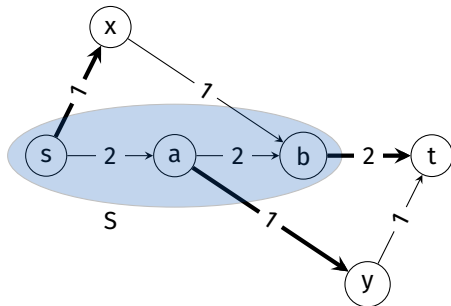
Proof of lemma 2

- ▶ lemma 2: for any flow network G , any flow f and any $s - t$ cut $V = S \cup T$, the value $|f|$ of flow f is at most the capacity of the cut, i.e.

$$|f| = f(S, T) \leq c(S, T)$$

- ▶ from lemma 1 we know $|f| = f(S, T)$
- ▶ consider **flows across $s - t$ cut $V = S \cup T$** , i.e. for $u \in S$ and $v \in T$ we have
 - ▶ $(u, v) \in E \implies f(u, v) \leq c(u, v)$
 - ▶ $(u, v) \notin E \implies f(u, v) = -f(v, u) \geq 0$
- ▶ we thus have

$$f(S, T) = \sum_{\substack{u \in S \\ v \in T \\ (u, v) \in E}} f(u, v) + \sum_{\substack{u \in S \\ v \in T \\ (u, v) \notin E}} f(u, v) \leq \sum_{\substack{u \in S \\ v \in T \\ (u, v) \in E}} f(u, v) \leq \sum_{\substack{u \in S \\ v \in T \\ (u, v) \in E}} c(u, v) = c(S, T) \quad \square$$



Notes:

Proof of lemma 3

1/2

- ▶ lemma 3: for any flow network G with source s and sink t , the following statements are equivalent:

- (1) f is maximum flow in G
- (2) no augmenting path exists in residual network G_f
- (3) $|f| = c(S, T)$ for some $s - t$ cut $V = S \cup T$

- ▶ to prove the equivalence of the statements we can prove:

- ▶ (1) $\implies$ (2)
- ▶ (2) $\implies$ (3)
- ▶ (3) $\implies$ (1)

- ▶ if augmenting path exists in G_f , then f can not be maximal, i.e. $\neg(2) \implies \neg(1)$, which is logically equivalent to (1) $\implies$ (2) $\rightarrow$ L07, contraposition

- ▶ (3) $\implies$ (1) follows from lemma 2 because:

- ▶ for any flow f and any $s - t$ cut we have $|f| \leq c(S, T)$ and thus $c(S, T) \geq |f|$
- ▶ if we have flow f and $s - t$ cut with $|f| = c(S, T)$ then flow must be maximal

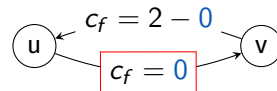
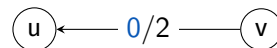
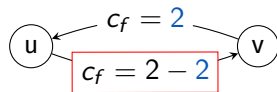
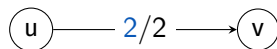
Notes:

Proof of lemma 3

- ▶ we now prove (2) $\implies$ (3), i.e. if no augmenting path exists in residual network G_f , then there exists $s - t$ cut S, T of G with $c(S, T) = |f|$
- ▶ proof by construction: for source node s define $S \subset V$ containing all nodes reachable from s in residual network G_f
 1. t is not in S , else there would be a path from s to t , i.e. we have a valid $s - t$ cut
 2. by construction, G_f can not contain edges from S to T (i.e. $c_f(u, v) = 0$ for $u \in S$ and $v \in T$) which implies
 - ▶ for $(u, v) \in E$ with $u \in S$ and $v \in T$: $f(u, v) = c(u, v)$
 - ▶ for $(u, v) \notin E$ with $u \in S$ and $v \in T$: $f(u, v) = 0$
 3. using lemma 1 we thus have

$$|f| = f(S, T) = \sum_{\substack{u \in S \\ v \in T \\ (u,v) \in E}} f(u, v) + \sum_{\substack{u \in S \\ v \in T \\ (u,v) \notin E}} f(u, v) = \sum_{\substack{u \in S \\ v \in T \\ (u,v) \in E}} f(u, v) = \sum_{\substack{u \in S \\ v \in T \\ (u,v) \in E}} c(u, v) = c(S, T) \quad \square$$

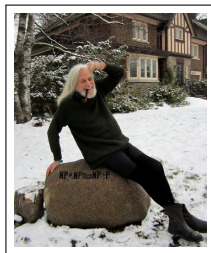
2/2



Notes:

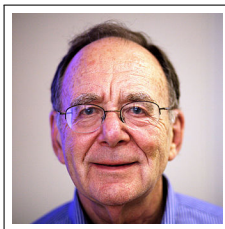
Edmonds-Karp algorithm

- ▶ we proved **correctness of terminal condition**, i.e. if Ford-Fulkerson method terminates then flow is maximal
- ▶ Ford-Fulkerson method is basis for flow maximization algorithms, but **not a complete algorithm by itself**
- ▶ poor choice of augmenting paths may result in algorithm not terminating
- ▶ we can use BFS to find **shortest paths along edges with residual capacity** → J Edmonds, RM Karp, 1972
- ▶ allows augmenting path to be found in $O(|E|)$
- ▶ length of augmenting paths increases monotonically, which ensures convergence to maximum flow value
- ▶ resulting **Edmonds-Karp algorithm** has overall runtime of $O(|V|^2|E|)$



Jack Edmonds
born 1934
John von Neumann
Prize in 1985

image credit: JeffAEdmonds,
CC-BY-SA



Richard M Karp
born 1935
Turing Award in 1985

image credit: Wikimedia Commons,
User: Rama, CC BY-SA 2.0

Notes:

Practice session

- ▶ we explore an implementation of the Edmonds-Karp algorithm in python
- ▶ we use the algorithm to **calculate a maximum flow** in our example network

19-01 Flow Maximization

July 3 2023

import pathpy as pp

✓ 00s

```
from copy import deepcopy
import numpy as np
```

```
def pp_nwk_from_mat(nodes, matrix):
    assert matrix.shape[0] == matrix.shape[1]
    n = matrix.shape[0]
    nwk = pp.Network(directed=True)
    for node in nodes:
        nwk.add_node(node)
    for i in range(n):
        for j in range(n):
            if matrix[i,j]>0:
                nwk.add_edge(nodes[i],nodes[j], capacity = matrix[i,j])
    return nwk
```

```
class FFA():
    def __init__(self, nodes, capacities, source, sink):
        self.nodes = nodes
        self.node2ind = {node:ind for ind,node in enumerate(nodes)}
        self.ind2node = {ind:node for ind,node in enumerate(nodes)}
        self.s = source
        self.t = sink
        self.n = len(nodes)
        self.capacities = capacities
        self.flow = np.zeros(shape = (self.n,self.n))
        self.flow_value = 0
        self.residual_network_capacities = deepcopy(capacities)
        self.residual_network = pp_nwk_from_mat(self.nodes, self.residual_network_capacities)
        self.augmenting_path = None
        self.augmenting_path_value = 0
```

practice session

see notebook 19-01 in gitlab repository at

→ https://gitlab2.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids2/

Notes:

In summary ...

- ▶ we considered flow maximization, a **combinatorial optimization problem** in graphs where edges carry **capacities**
- ▶ we showed how we can use the **Ford-Fulkerson method** to calculate maximum flows
- ▶ basis for flow maximization algorithms that have **important applications in various domains**
- ▶ can be extended to graphs with **multiple source and sink nodes** → self-study questions
- ▶ **max-flow min-cut theorem** clarifies that flow maximization and (edge) cut minimization are **dual versions of the same optimization problem**

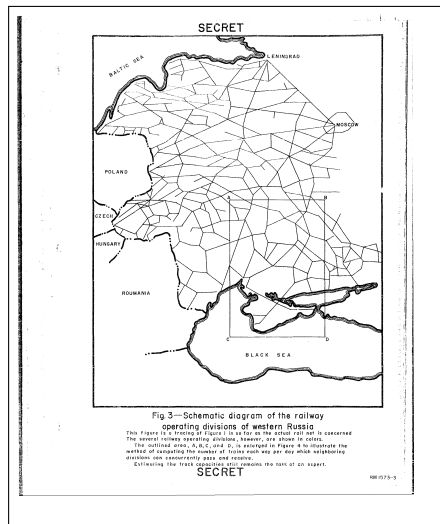


image credit: adapted from → TE Harris, FSRoss, 1955

Notes:

Self-study questions

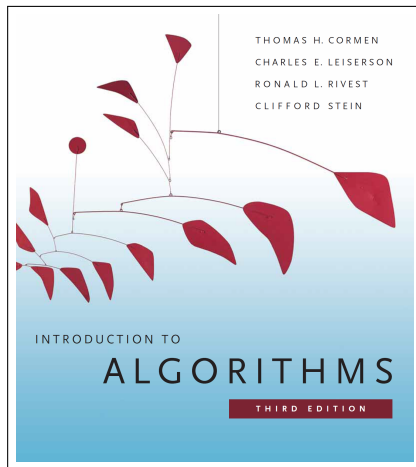
1. What is a flow network? How do flows f relate to the edges in the graph and their capacities?
2. Prove the properties of net flows given in the box on slide 5.
3. Explain how the construction of the residual network avoids local optima in the Ford-Fulkerson method.
4. Construct an example network, where you can choose augmenting paths in the residual network such that the Ford-Fulkerson method never terminates.
5. Investigate how the Edmonds-Karp algorithm can be applied to flow networks with multiple source and sink nodes.
6. What is the max-flow min-cut theorem and how is it related to the Ford-Fulkerson method?
7. Summarize the proof idea behind the max-flow min-cut theorem.
8. Investigate how we can use the Edmonds-Karp algorithm to calculate maximum matchings in bipartite graphs.

Notes:

References

reading list

- ▶ TE Harris, FS Ford : **Fundamentals of a method for evaluating rail net capacities**, Rand Corporation, Research Memorandum, 1956
- ▶ LR Ford and DR Fulkerson: **Maximal flow through a network**, Canadian Journal of Mathematics, 1956
- ▶ LR Ford: **Network flow theory**, Technical Report P-923, The Rand Corporation, 1956
- ▶ EA Dinic: **Algorithm for solution of a problem of maximum flow in a network with power estimation**, Soviet Mathematics - Doklady, 1970
- ▶ J Edmonds and RM Karp: **Theoretical improvements in algorithmic efficiency for network flow problems**, Journal of the ACM, 1972
- ▶ TH Cormen, CE Leiserson, RL Rivest: **Introduction to Algorithms**, MIT press, 2022



Notes: